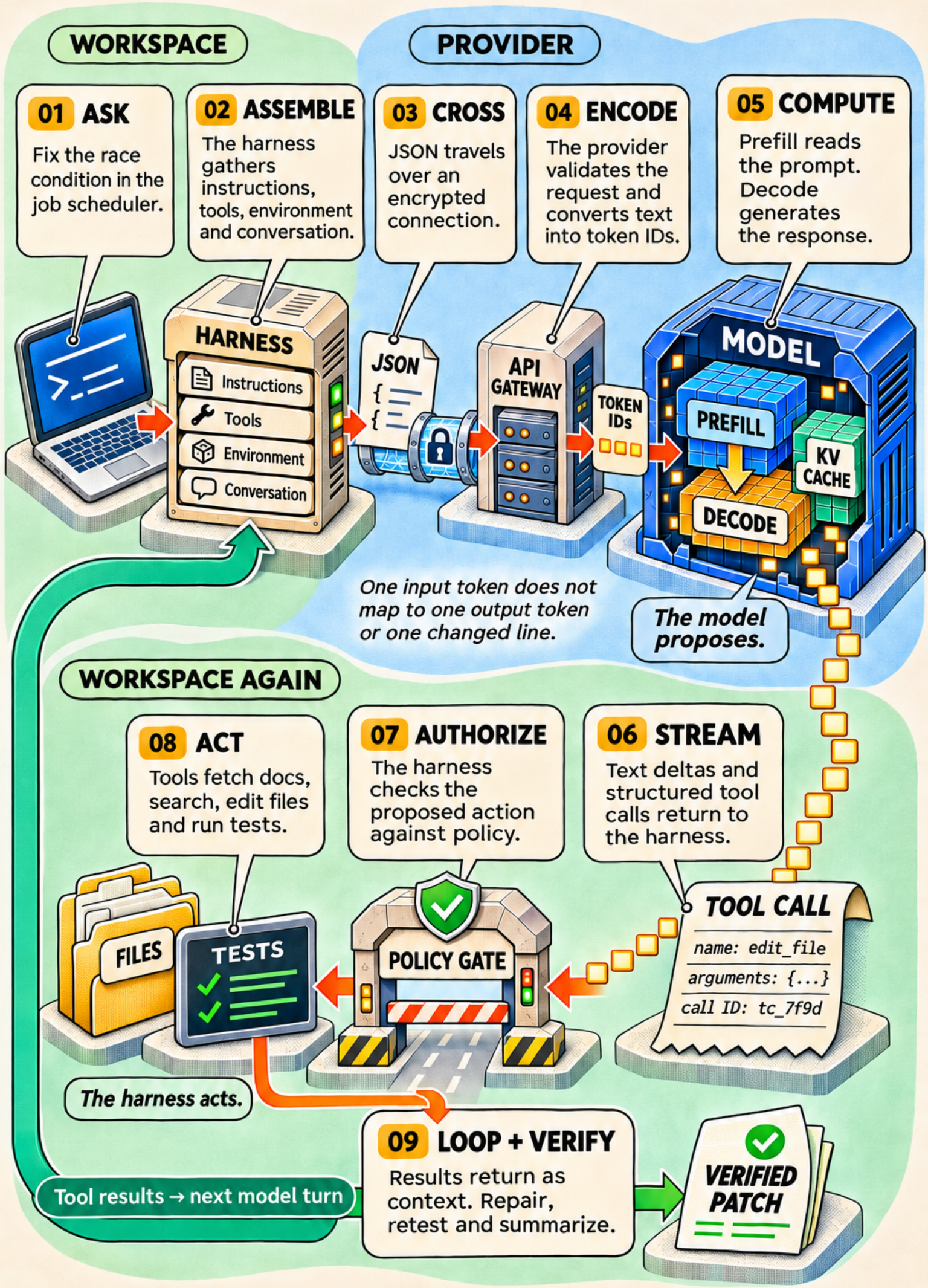
HOW A CODING AGENT WORKS



THREE HARNESSES, ONE LOOP

CLAUDE CODE

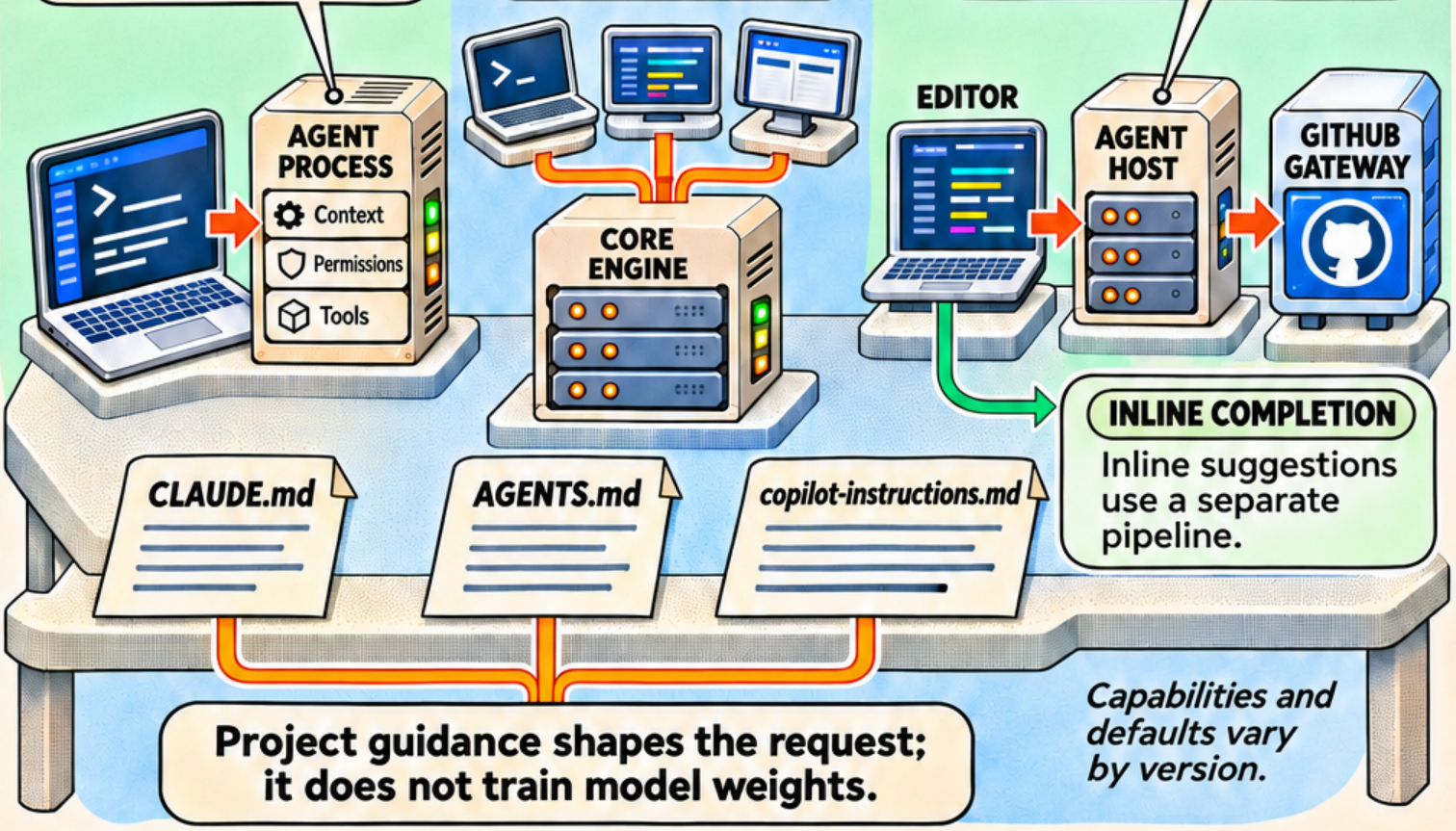
An agent process coordinates context, permissions and tools.

CODEX

A core engine serves terminal, editor and desktop clients.

GITHUB COPILOT

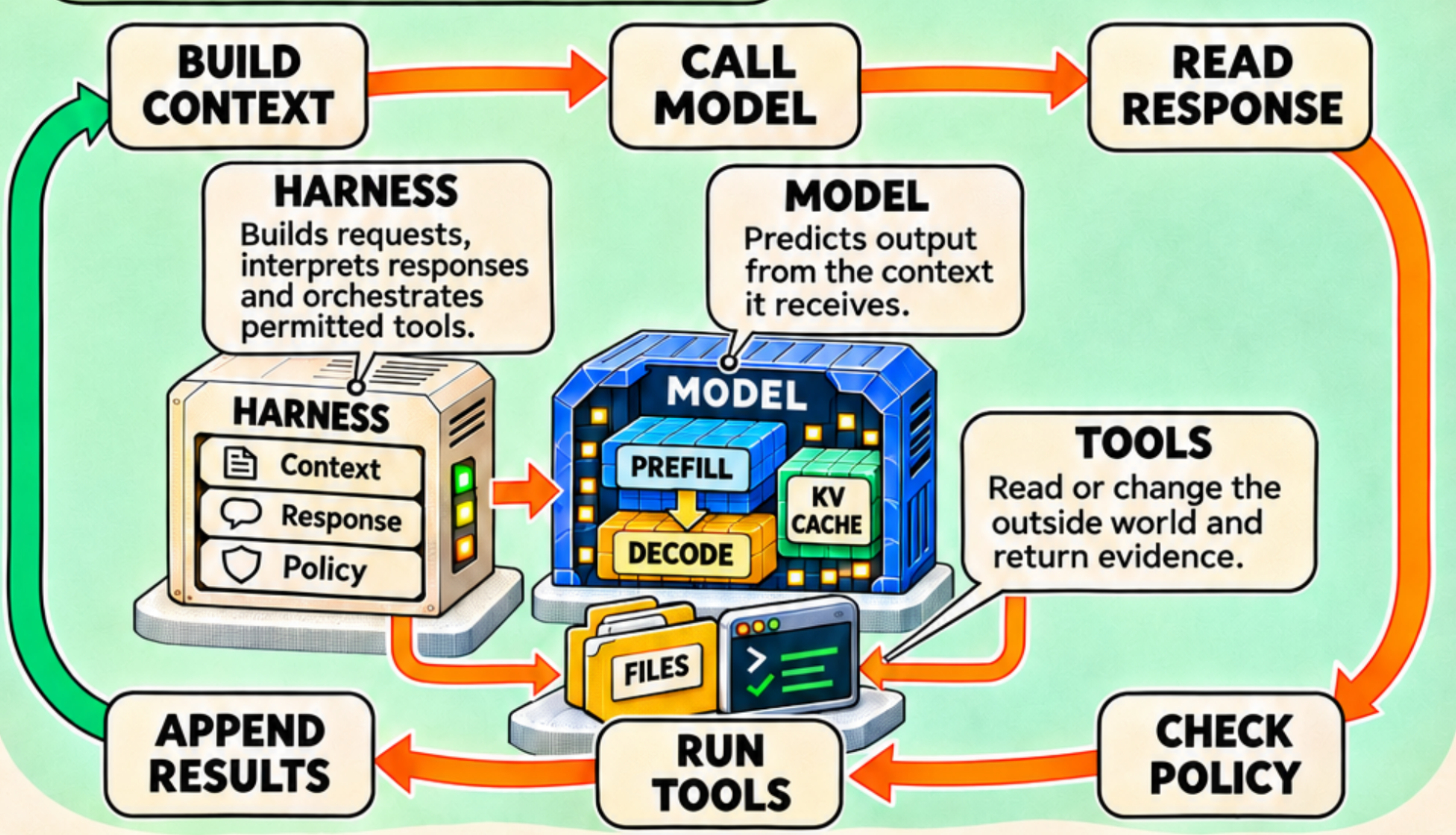
The editor connects to an agent host. Model requests go through GitHub's service.



Project guidance shapes the request; it does not train model weights.

Capabilities and defaults vary by version.

COMMON CONTROL LOOP



BUILDING THE MODEL'S CONTEXT

MCP CONNECTIONS

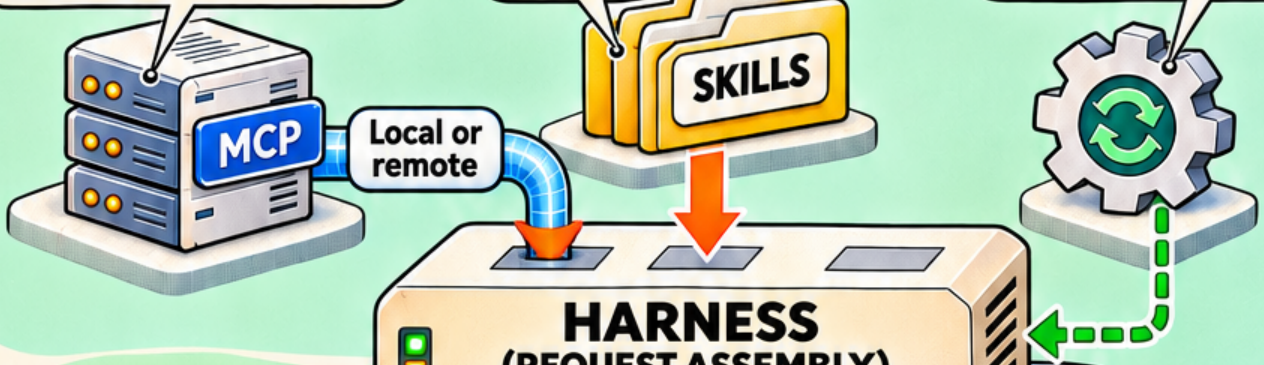
Servers expose tools and resources through defined interfaces.

SKILLS

Task-specific instructions are loaded when useful.

HOOKS

Run at configured events; some hooks add context.



HARNESS (REQUEST ASSEMBLY)

STABLE PREFIX

Keep instructions and tool definitions consistent.

An identical prefix may reuse earlier computation.

INSTRUCTIONS

Harness rules, project guidance and task constraints.

TOOLS

Names, descriptions and argument schemas.

A schema describes an action. Policy decides whether it may run.

ENVIRONMENT

Working directory, platform and available capabilities.

GROWING TAIL

Append new messages and tool results.

CONVERSATION

User messages, model replies, tool calls and results.

CONTEXT IS SELECTED

Files enter through attachments, editor context or tool reads. The model does not automatically see the whole disk.

STRUCTURED REQUEST

- messages
- tools
- settings

ACROSS THE WIRE

CLIENT **PROVIDER** Conceptual flow; order varies.

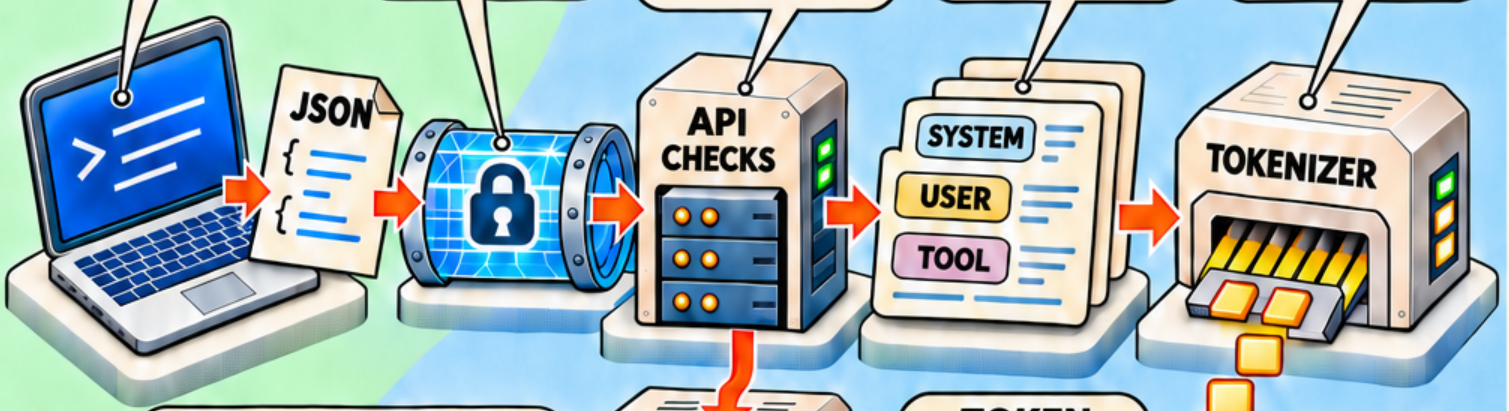
JSON REQUEST
The client sends structured text, tool definitions and settings.

TLS TRANSPORT
Encrypted bytes travel over the network.

API CHECKS
Check credentials, request shape, rate limits and capacity.

CHAT TEMPLATE
Mark roles, messages and tool boundaries.

TOKENIZER
Map text pieces to vocabulary IDs.



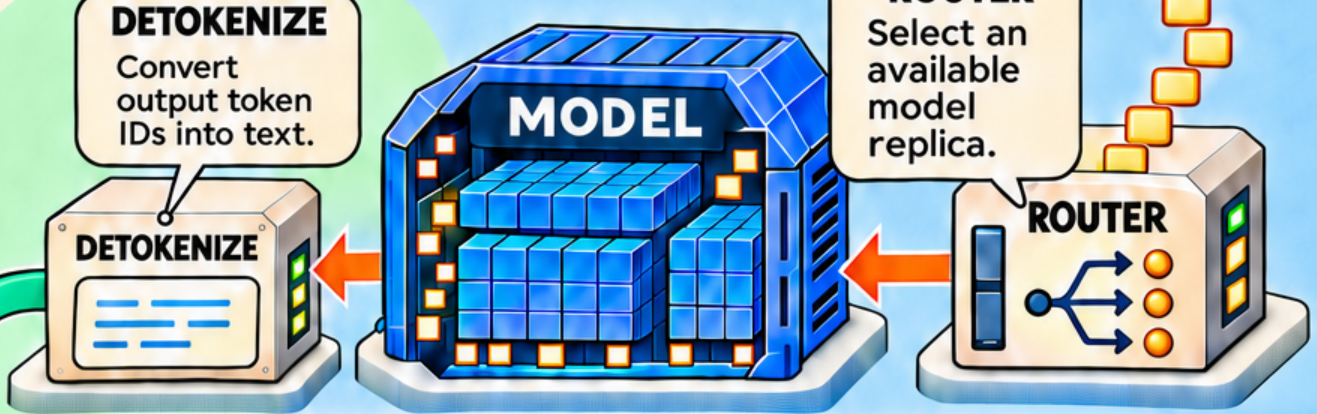
CAPACITY UNAVAILABLE
Queue the request or return an error for the harness to handle.

TOKEN BUDGET
Check the rendered token count.

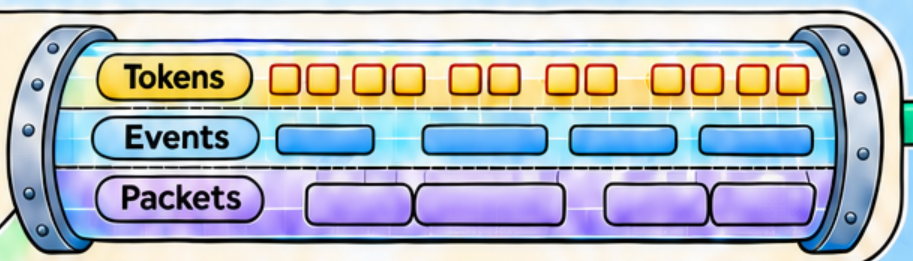
TOKEN IDS

ROUTER
Select an available model replica.

DETOKENIZE
Convert output token IDs into text.



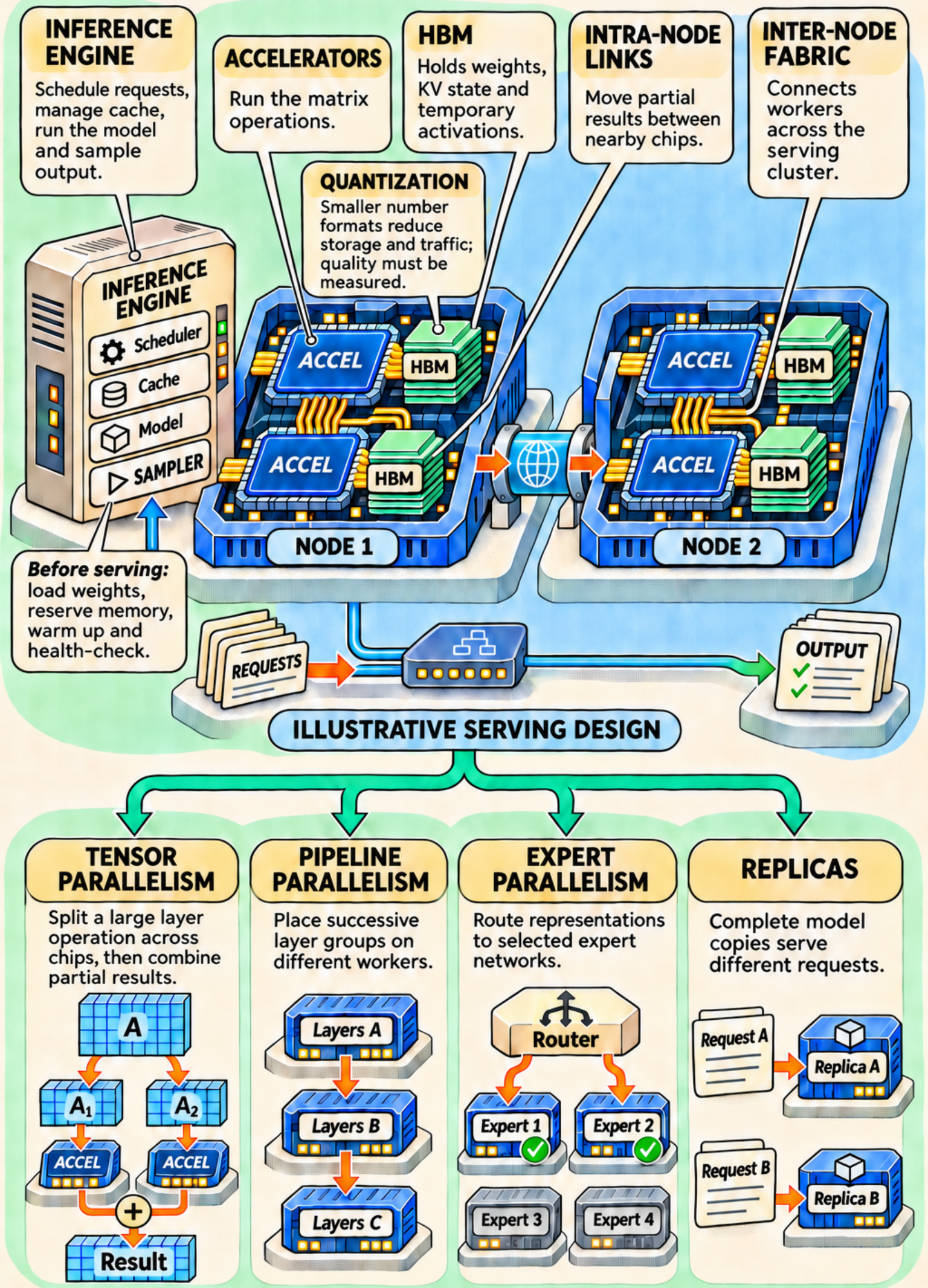
These boundaries do not line up one to one.



REASSEMBLE
Collect complete tool-call arguments before execution.

STREAMED EVENTS
Text deltas and structured items arrive incrementally.

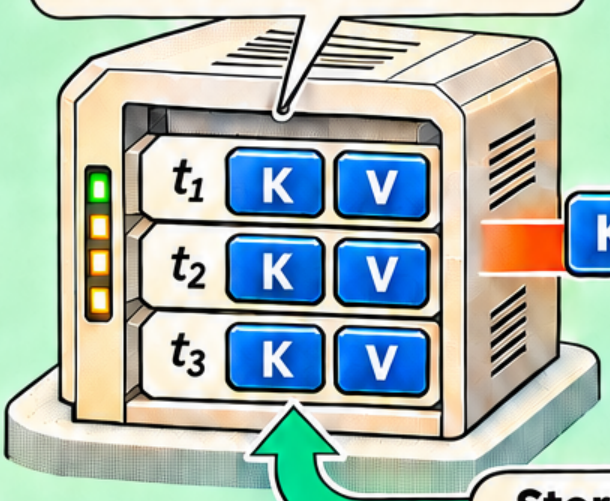
ONE MODEL, MANY ACCELERATORS



HOW THE KV CACHE SAVES WORK

KV CACHE

Store keys and values already computed for earlier tokens.



ATTENTION

New queries reuse earlier keys and values to gather context.

K V

Store new K and V

PAGED ALLOCATION

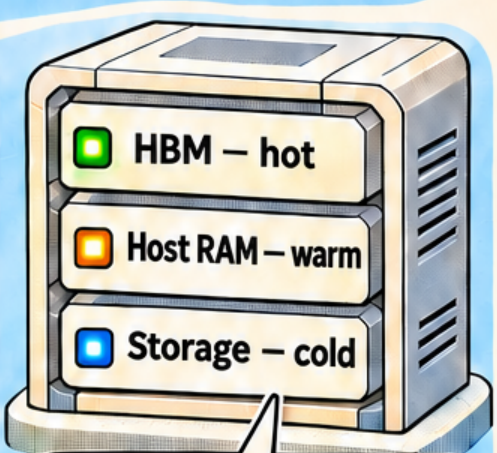
Logical blocks can live in non-adjacent physical slots.

Logical blocks (sequence order)

Physical slots (memory)

Block Table
(logical → physical)

A	→ 3
B	→ 1
C	→ 4



Some serving designs move cached blocks between memory tiers.

PREFIX REUSE

Compatible requests can reuse matching, eligible prefix blocks.

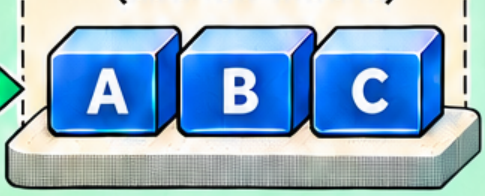
Request 1 A B C D

Request 2 A B C E

REUSE

COMPUTE

Shared prefix
(cached blocks)



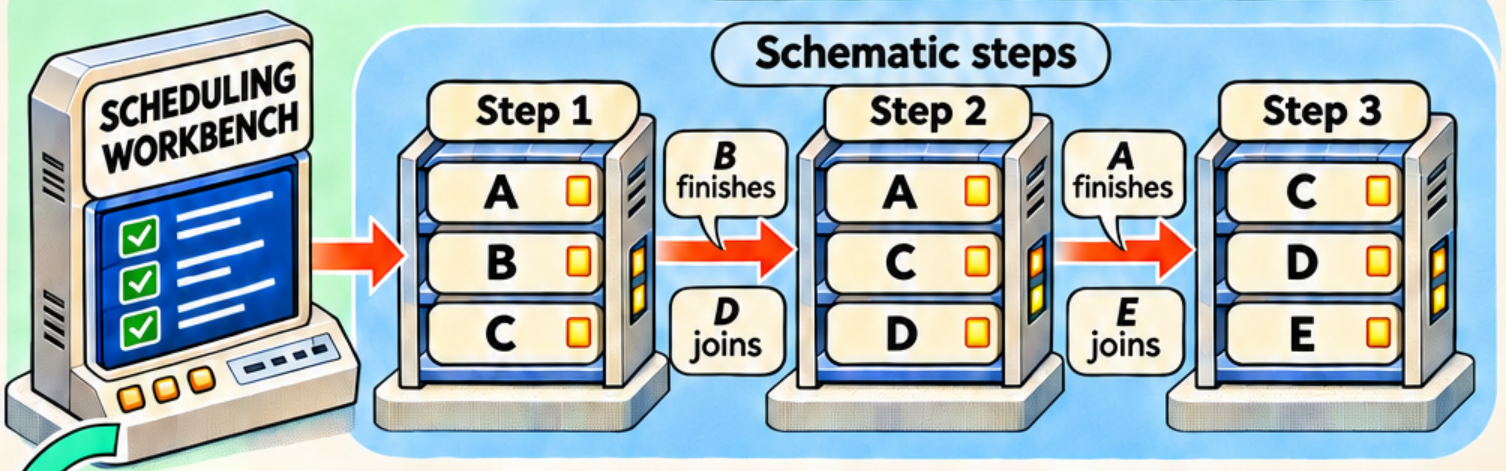
Full-attention KV needs grow with retained tokens and the model's KV dimensions.

A cache hit reuses computation. It is not conversation knowledge or a change to learned weights.

SHARING THE INFERENCE ENGINE

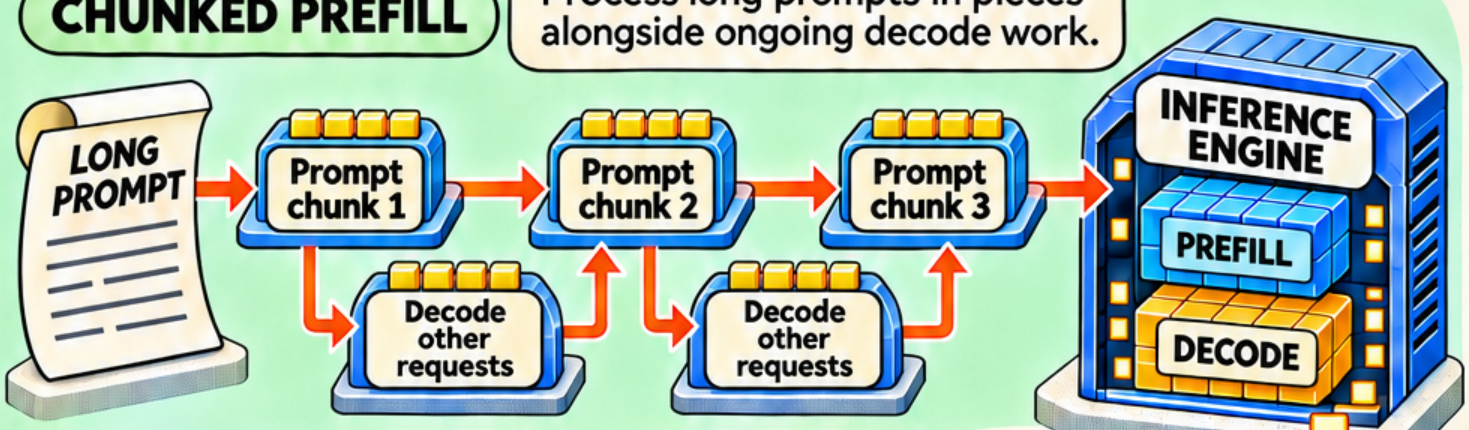
CONTINUOUS BATCHING

A finished sequence leaves; a waiting request can enter the next step.



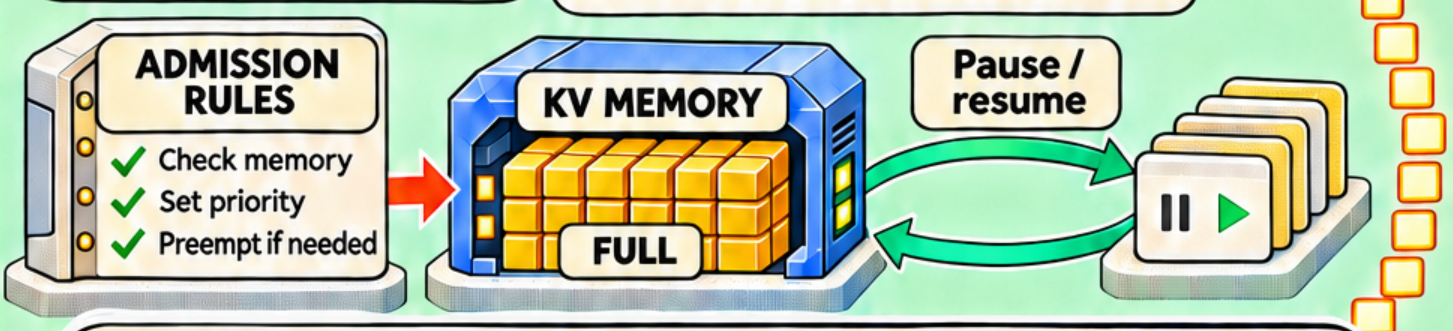
CHUNKED PREFILL

Process long prompts in pieces alongside ongoing decode work.



MEMORY PRESSURE

Admission rules and preemption control which sequences can run.

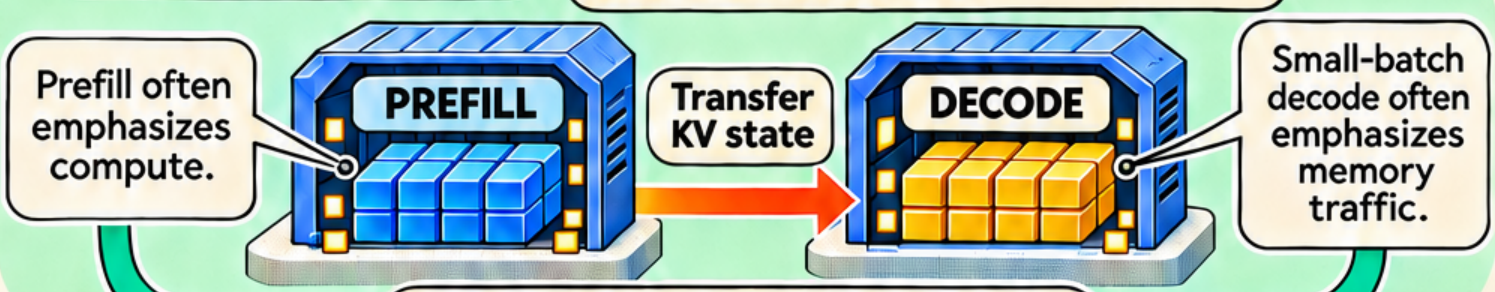


Step time ≥ max(compute time, memory-transfer time)

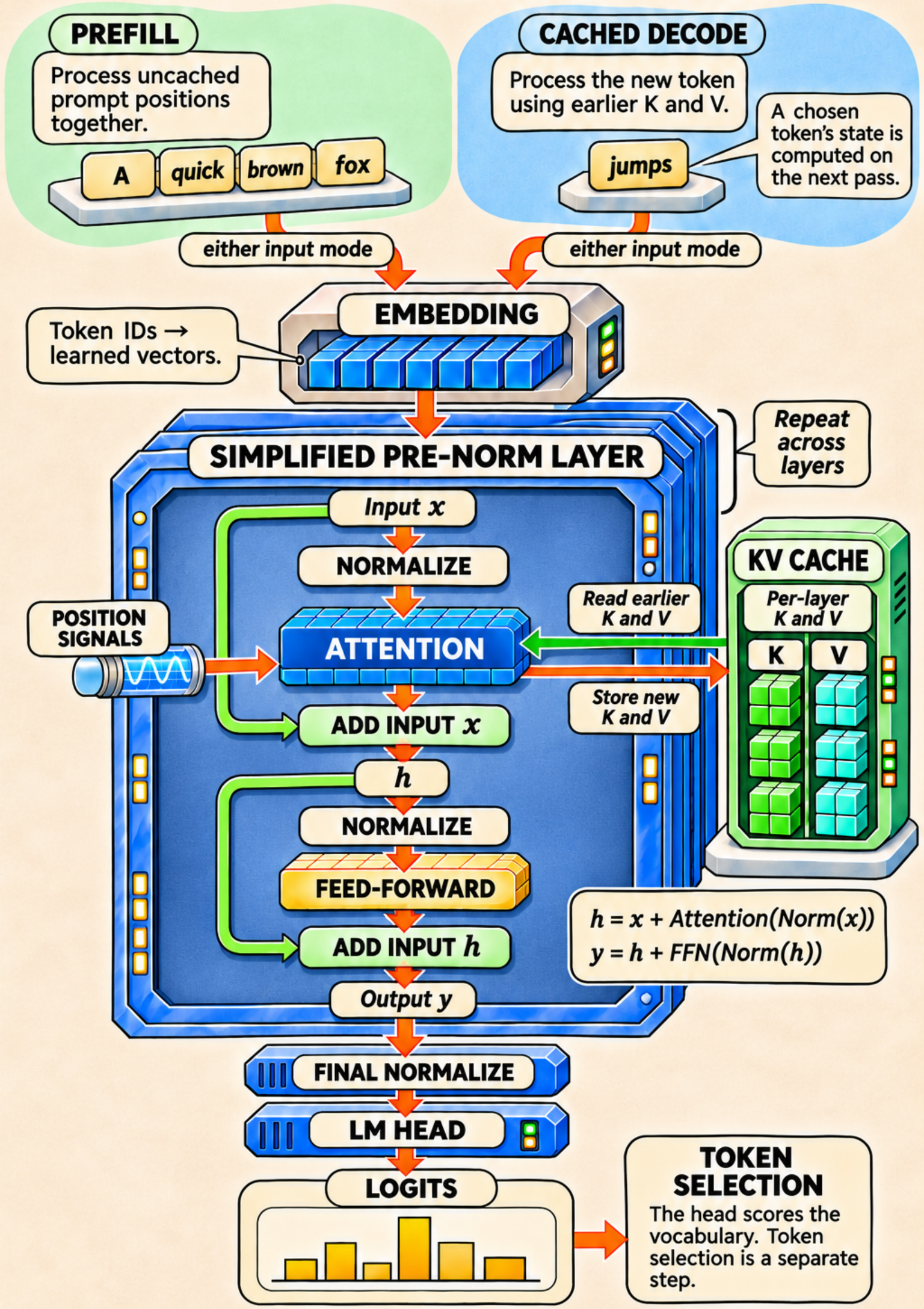
Network and scheduling add overhead. Workload can shift the bottleneck.

SEPARATED POOLS

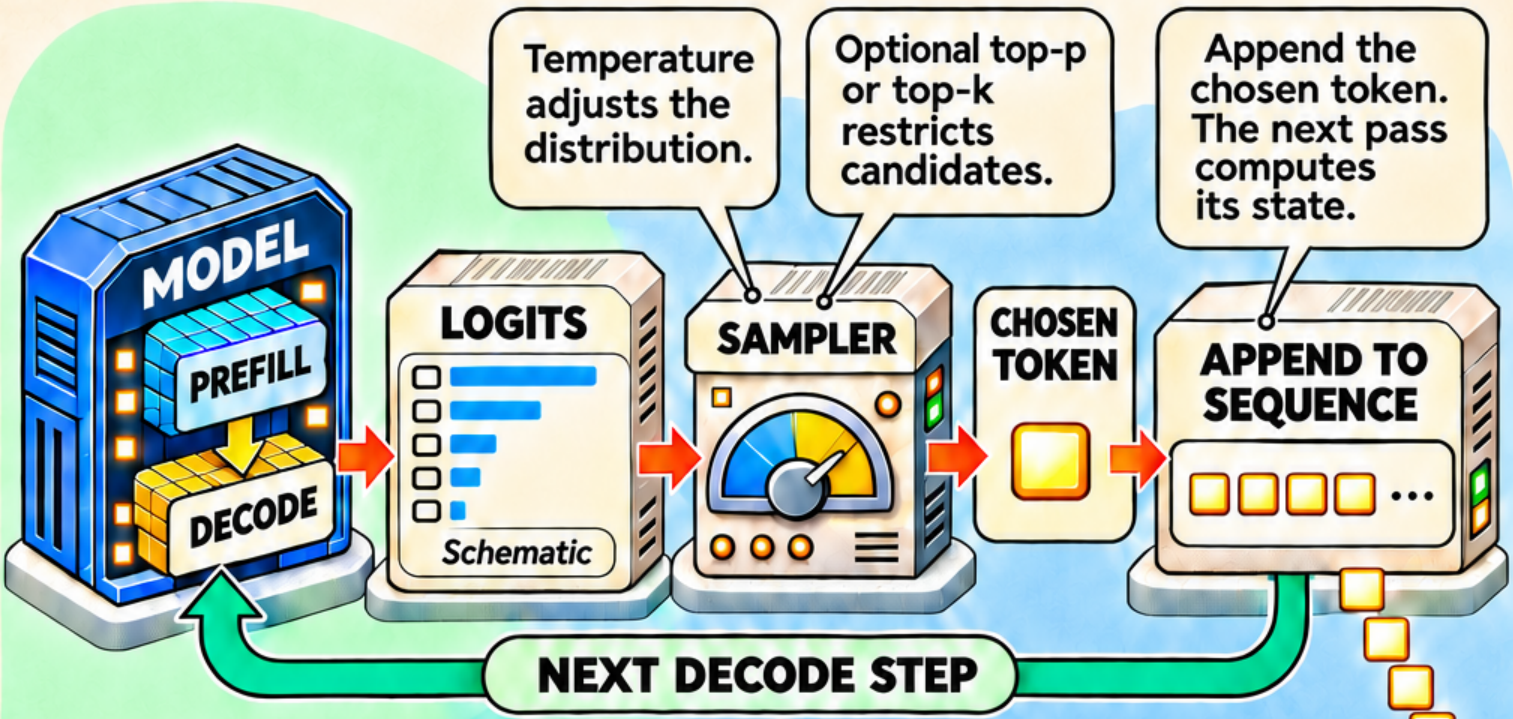
Some designs run prefill and decode on different workers.



INSIDE A FORWARD PASS

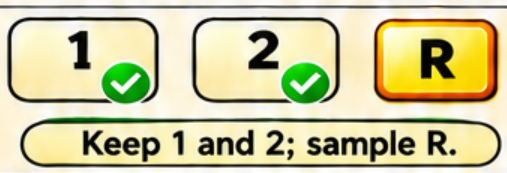


FROM SCORES TO TOOL CALLS



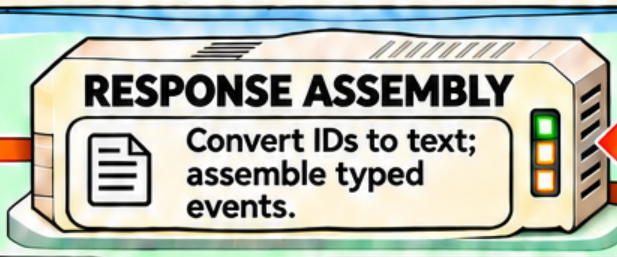
SPECULATIVE DECODING

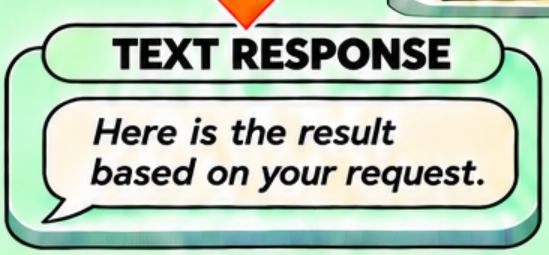
Exact acceptance and correction preserve the target distribution.



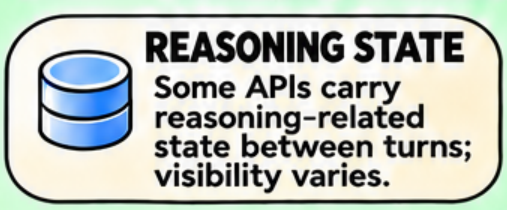
If all drafts pass, sample one extra token.

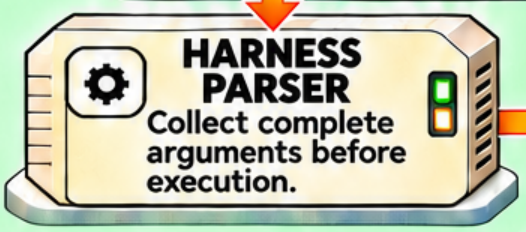
Use the adjusted distribution at the first rejection.



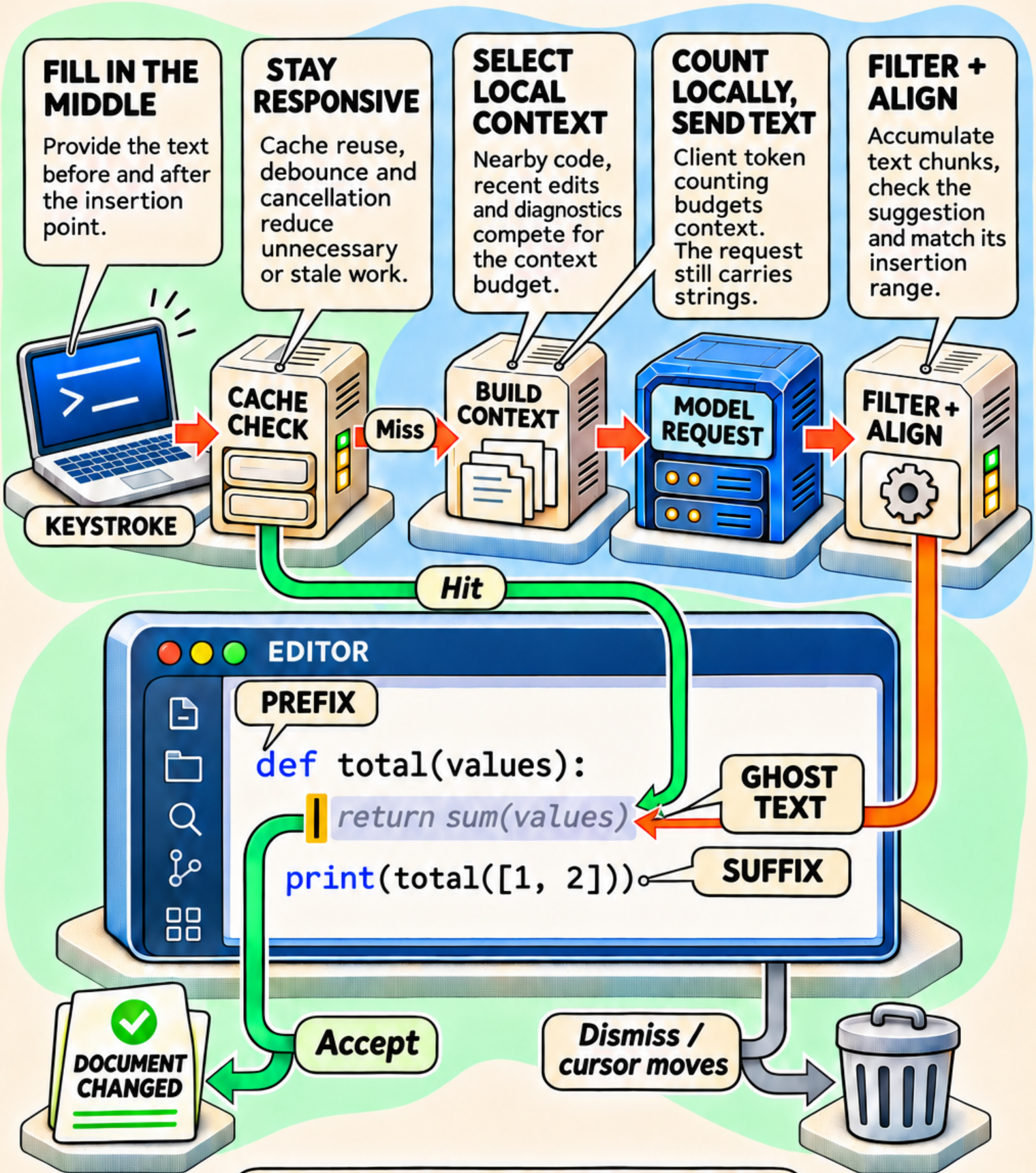


Valid syntax does not grant authority.

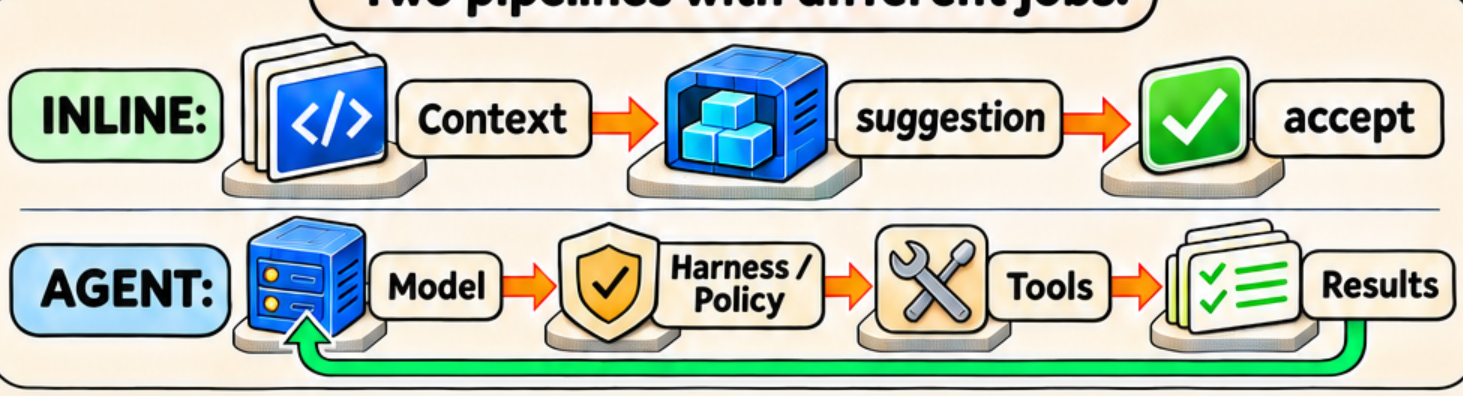




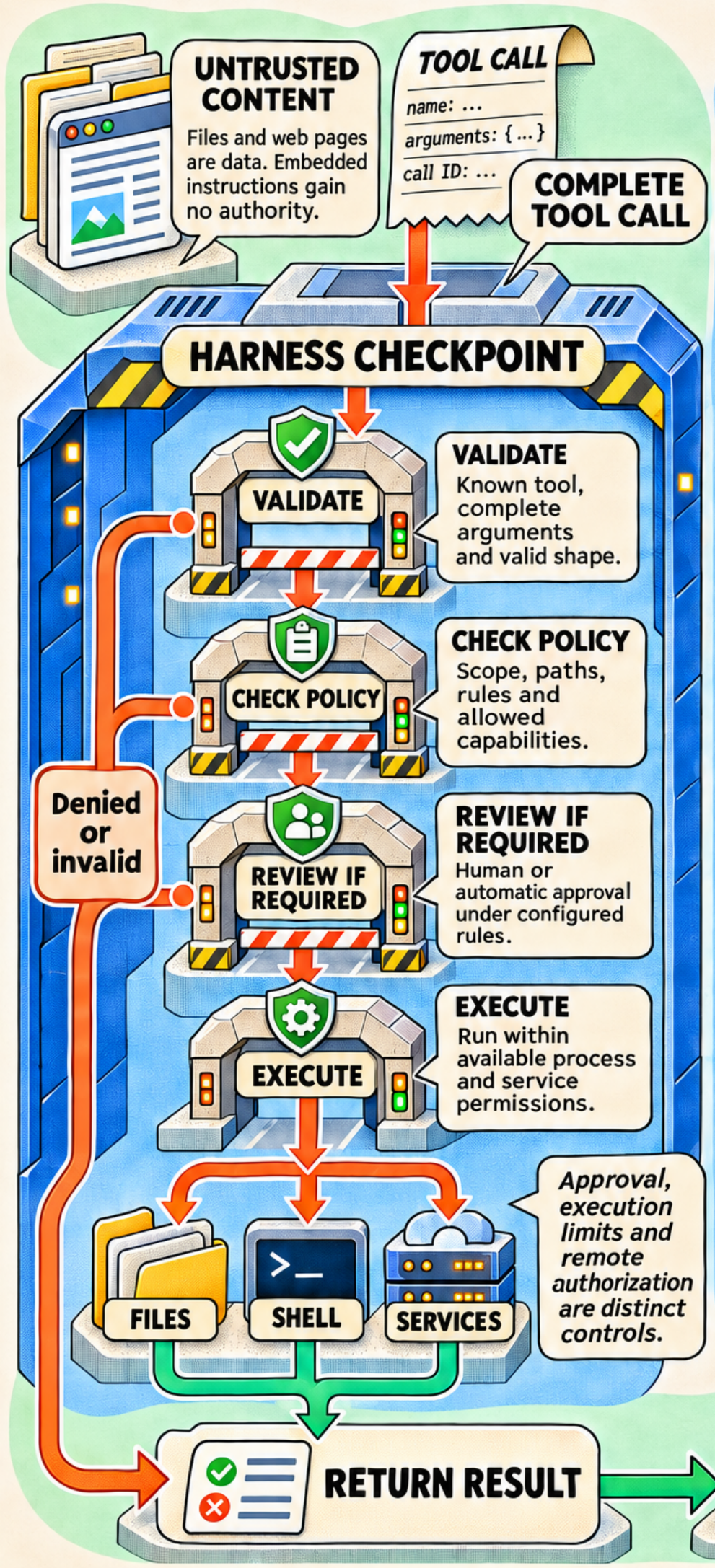
CLASSIC INLINE COMPLETION



Two pipelines with different jobs.



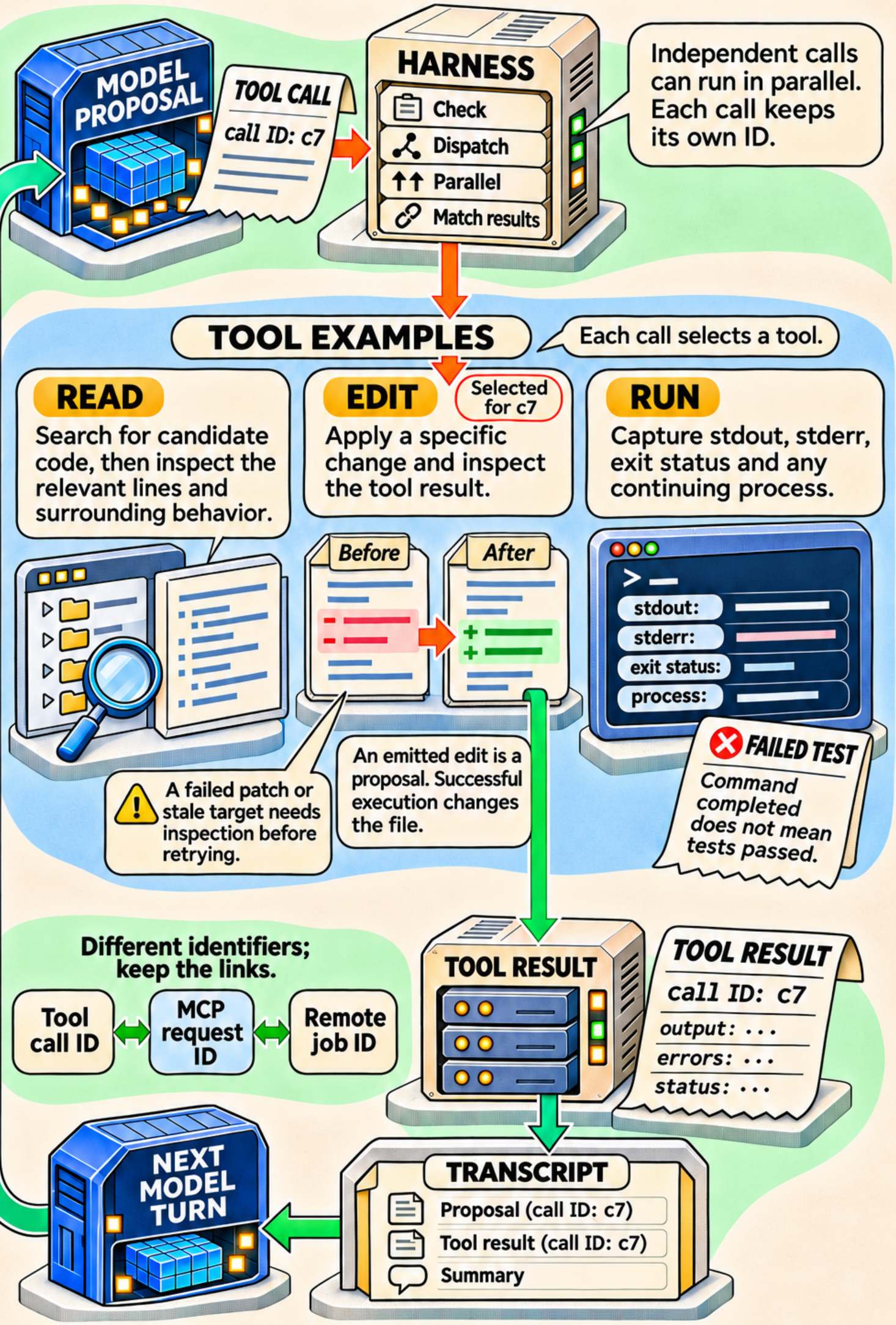
FROM INTENT TO AUTHORITY



AUTHORITY STACK

1. **INSTRUCTIONS** — What should be done?
2. **PROVIDER SAFETY** — What may the model emit?
3. **HARNESS POLICY** — Which actions are allowed?
4. **REVIEW** — Who must confirm?
5. **EXECUTION BOUNDARY** — Where can the process reach?
6. **REMOTE AUTHORIZATION** — What may credentials access?
7. **DATA FLOW** — What data may enter or leave?

TOOLS TURN REQUESTS INTO EVIDENCE



SEVEN KINDS OF MEMORY

HARNESS (SESSION)

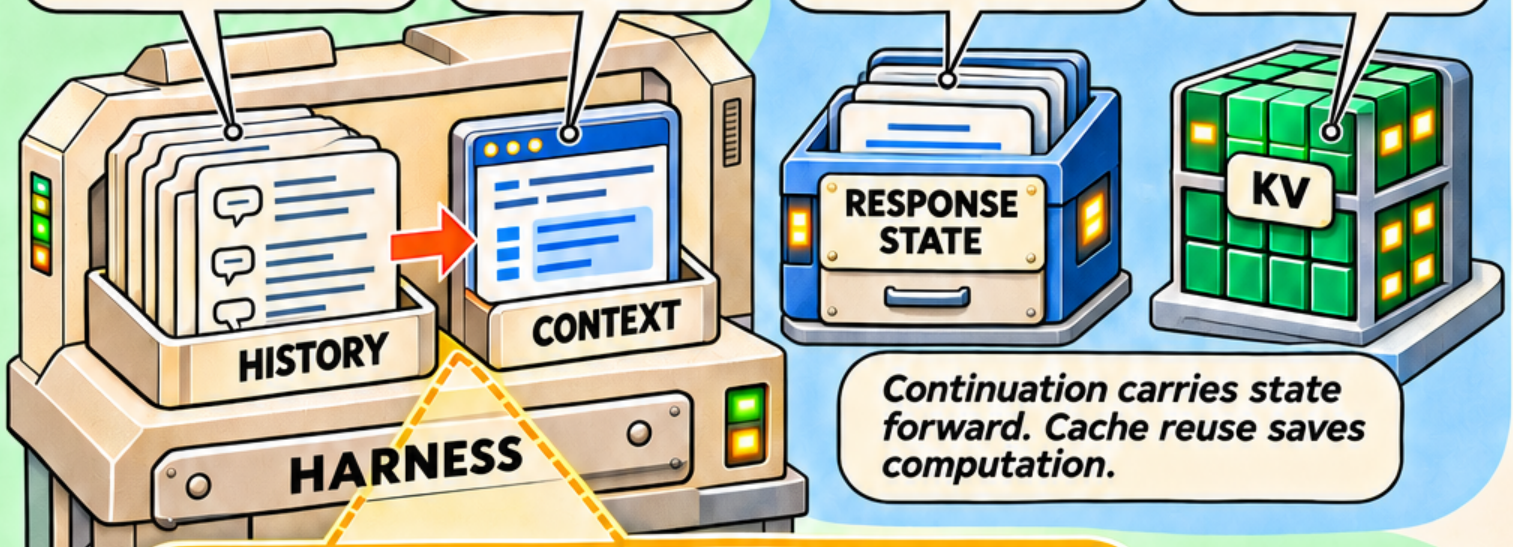
1 CONVERSATION HISTORY
Messages, tool calls and results saved for the session.

2 CURRENT CONTEXT
The selected material the model sees on this request.

PROVIDER

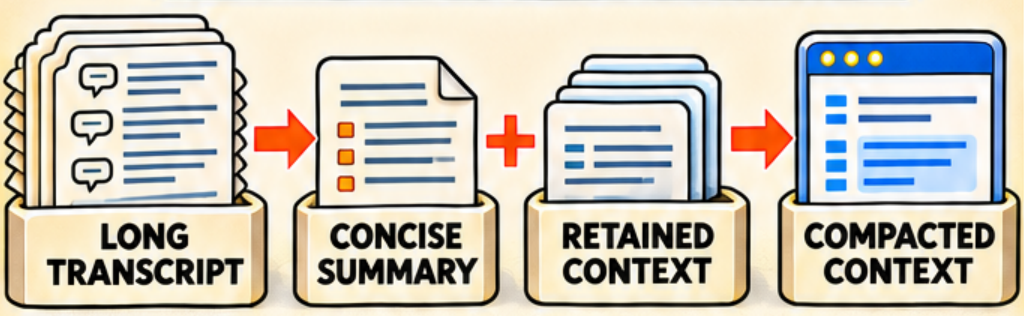
3 PROVIDER CONTINUATION
Stored response state used to continue a conversation.

4 INFERENCE CACHE
Computed keys and values reused during inference.



Continuation carries state forward. Cache reuse saves computation.

ONE COMPACTION APPROACH



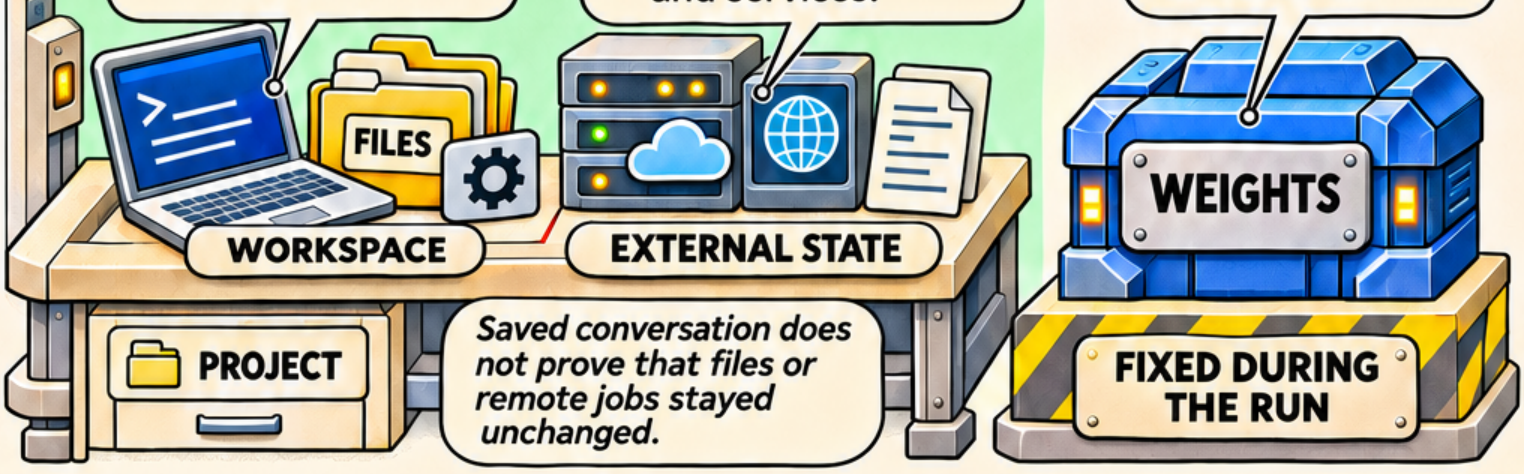
Preserve the goal, constraints, evidence, failures and next step.

Saved history can be longer than the context sent this turn.

5 WORKSPACE
Files, builds and running processes.

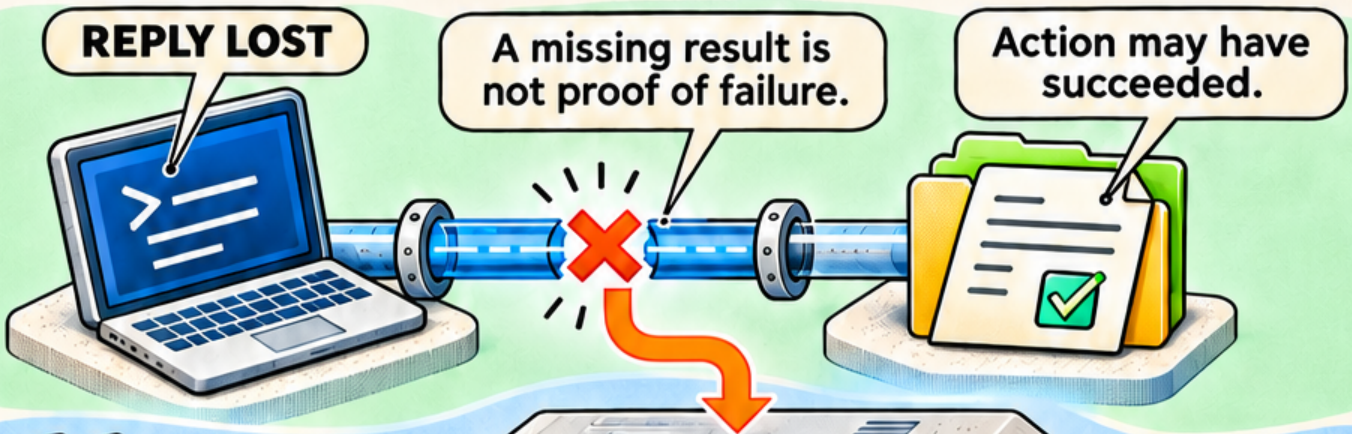
6 EXTERNAL STATE
Branches, remote jobs, documents and services.

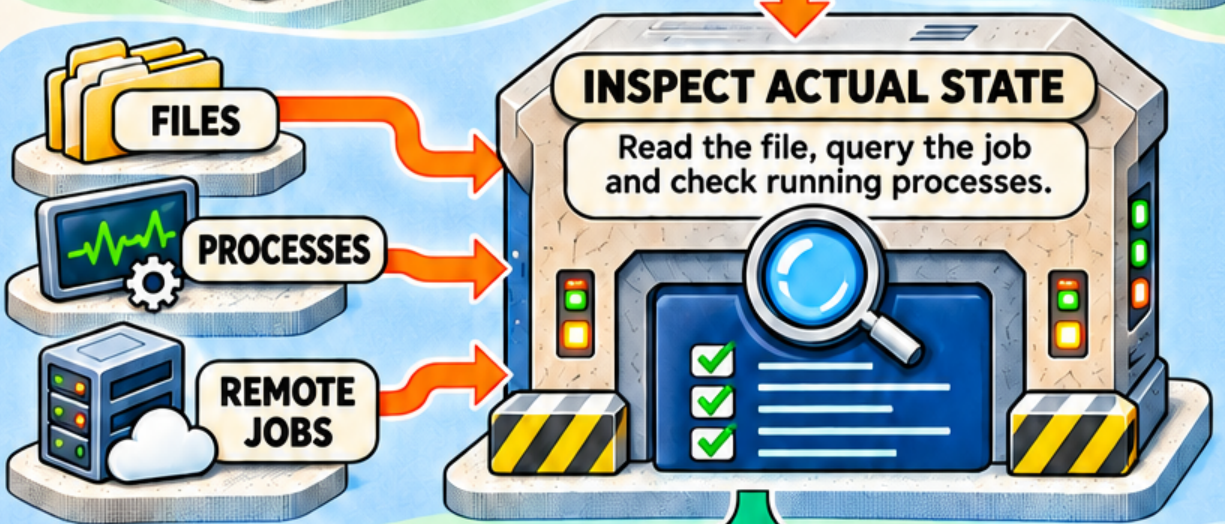
7 MODEL PARAMETERS
Learned weights; unchanged by this session.



Saved conversation does not prove that files or remote jobs stayed unchanged.

RECOVERING FROM AN UNKNOWN OUTCOME





CONFIRMED APPLIED
Continue from the observed state.

CONFIRMED NOT APPLIED
Retry when appropriate.

Check revisions. Use safe-to-repeat operations. Bound retries.

STILL UNCERTAIN
Reconcile and report uncertainty.

CONTROL ACTIONS

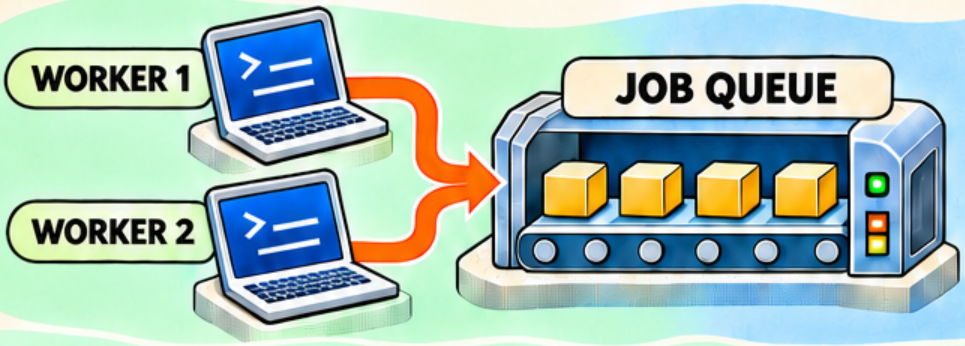
STEER
Change the plan for future work.

CANCEL
Stop a request or stream; a tool may still be running.

TERMINATE
Stop the actual process and verify it ended.

ROLL BACK
Undo a completed effect through a separate operation, where possible.

A FIX THROUGH TEN TURNS



Illustrative scheduler example

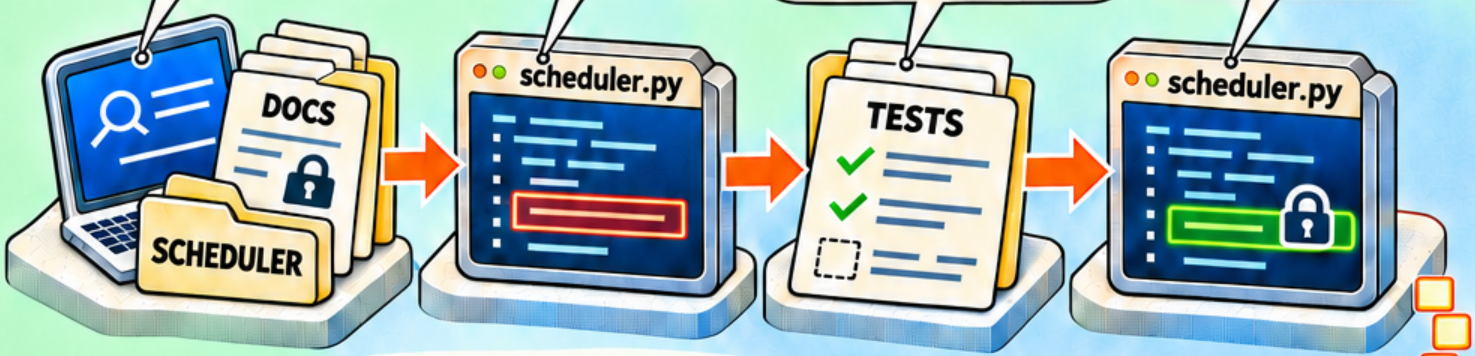
Two workers share a queue; coordination matters.

01 SEARCH + DOCS
Find scheduler code and read the locking documentation.

02 READ CODE
Inspect the shared queue and the unprotected operation.

03 INSPECT TESTS
Read existing tests and identify missing concurrency coverage.

04 PATCH
Protect the shared operation with a lock.

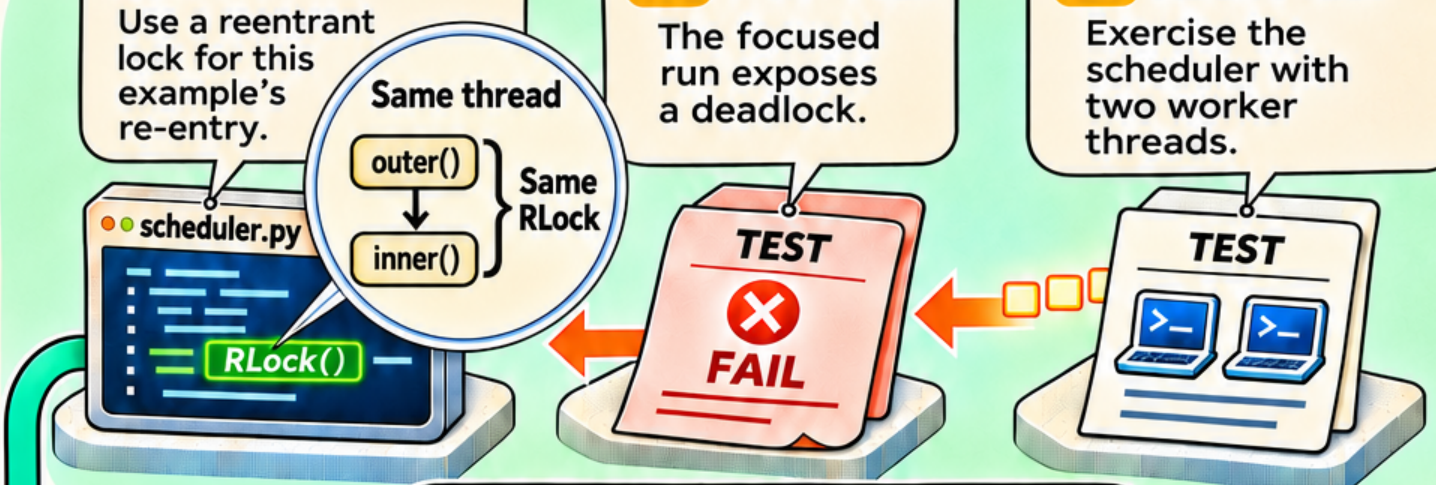


Tool feedback changes the next action.

05 ADD A TEST
Exercise the scheduler with two worker threads.

06 TEST FAILS
The focused run exposes a deadlock.

07 REPAIR
Use a reentrant lock for this example's re-entry.



Passing tests are evidence, not proof.

08 RETEST
Run the focused checks again.

09 BROADER CHECKS
Run the full suite against the updated files.

10 SUMMARIZE
Present the diff, test evidence and remaining limits.

WHAT FINISHED MEANS

Checks must match the final revision.

Report what changed, what was tested and what remains uncertain.

Preserve the links between calls, revisions and results.

FINAL FILES r3

DIFF
Files changed
Actual edits
Why they help

TEST EVIDENCE
Commands run
Observed results
Revision tested: r3

KNOWN LIMITS
Skipped checks
Unresolved risks
Next steps

Passing tests provide evidence; they do not prove correctness.

REVIEWABLE HANDOFF

Separate operations; each needs authority.

PATCH
A reviewable change.

COMMIT
A recorded repository revision.

PULL REQUEST
A proposed change for review.

MERGE
A change to the shared branch.

DEPLOY
A change to a running environment.

SCHEMATIC USAGE

Uncached input
Cached input
Output

Total usage includes every call and retry.

ELAPSED TIME
Parallel elapsed time follows the critical path.

RUN OUTCOME

- Complete
- Needs input
- Cancelled
- Budget exhausted
- Blocked